

Formulação do Problema: Cobertura Exata de Arestas com Subcaminhos Limitados

Tabela de Variáveis Utilizadas	1
Algoritmo de Construção do Conjunto P	1
Complexidade:	2
• Função gerar_subcaminhos(s, K)	2
• Função gerar_conjuntoP(P, S)	3
Variáveis de Decisão:	3
Parâmetros Auxiliares:	3
Função Objetivo:	3
Restrições:	3
Cobertura Exata de Arestas com Subcaminhos Limitados	1

Tabela de Variáveis Utilizadas

- $G=(V,E)$: grafo direcionado.
- $S \subseteq \text{Paths}(G)$: conjunto de caminhos válidos na rede.
- $K \in \mathbb{N}$: número máximo de arestas por subcaminho.
- P : conjunto de todos os subcaminhos possíveis de caminhos em S , com no máximo K arestas.
 - Cada $p \in P$ é tal que $p \subseteq s$ para algum $s \in S$ e $|p| \leq K$.
- S' : Conjunto de subcaminhos de G selecionados para cobrir o grafo.

Algoritmo de Construção do Conjunto P

```
# Esta função gera todos os subcaminhos de tamanho  $\leq K$  de um caminho s
def gerar_subcaminhos(s, K):
    subcaminhos = []

    # Para cada aresta do caminho s
    for i in range(len(s)):
        ## O subcaminho se inicia na aresta i e finaliza na aresta j,
        ## portanto, a segunda iteração se inicia em i + 1, garantindo que o
```

```

subcaminho tenha ao menos uma aresta (Subcaminhos não-vazios)
    ## O subcaminho terá no máximo K aresta. Caso o caminho s tiver
    menos do que K arestas, o subcaminho s' se encerrará na última aresta do
    caminho s (0 +1 é porque a função range em Python é exclusivo no segundo
    parâmetro)
    for j in range(i + 1, min(i + K + 1, len(s) + 1)):
        ## Forma o subcaminho
        sub = tuple(s[i:j])
        ## Adiciona o subcaminho formado na lista de subcaminhos do
    caminho s
        subcaminhos.append(sub)
    return subcaminhos

P = set()

## Esta função gerará todos os subcaminhos válidos de tamanho máximo K
de um conjunto de caminhos S
def gerar_conjuntoP(P,S):
    for s in S:
        ## Adiciona, para cada caminho s, os respectivos subcaminhos gerados
        ## A propriedade de P ser um conjunto garante que não haja repetição
        P.update(gerar_subcaminhos(s, K))

```

Complexidade:

- Função gerar_subcaminhos(s, K)

```

def gerar_subcaminhos(s, K):
    subcaminhos = []
    for i in range(len(s)): → Executa |s| vezes
        for j in range(i + 1, min(i + K + 1, len(s) + 1)): → Limite
superior é K iterações por i
            sub = tuple(s[i:j]) → Custo para criar um tupla de tamanho K
é O(K)
            subcaminhos.append(sub)
    return subcaminhos

```

$$\text{Tempo total} = O(n \cdot K^2)$$

- Função gerar_conjuntoP(P, S)

```
def gerar_conjuntoP(P, S):
    for s in S: → Executa |S| vezes
        P.update(gerar_subcaminhos(s, K)) → Custa  $O(|s|K^2)$ 
```

$$\text{Tempo total} = O(m \cdot n \cdot K^2)$$

Obs.: Supomos que o tamanho médio dos caminhos é $|s|$.

Variáveis de Decisão:

- $x_p \in \{0,1\} \forall p \in P$: $x_p=1$ se o subcaminho p é selecionado para fazer parte de S' , 0 caso contrário.

Parâmetros Auxiliares:

- $\delta_{e,p} \in \{0,1\}$: $\delta_{e,p}=1$ se a aresta $e \in E$ pertence ao subcaminho $p \in P$, 0 caso contrário.

Função Objetivo:

Minimizar o número total de subcaminhos usados:

$$\min \sum_{p \in P} x_p$$

Restrições:

1. **Cobertura exata de cada aresta:**

Cada aresta do grafo deve ser coberta **por exatamente um** subcaminho:

$$\sum_{p \in P} \delta_{e,p} \cdot x_p = 1 \quad \forall e \in E$$

2. **Capacidade máxima de cada subcaminho:**

Cada subcaminho em P tem no máximo K arestas por construção.